HPC monitor & assistant

Wentao Zhou, Guangyi Xu, Jinwei Zhou

4/25

Motivation

- HPC clusters are shared resources; users waste time checking node availability manually, especially availability for different hardware.
- No unified view across multiple clusters (CS & Rivanna)
- Users often submit jobs to busy partitions, leading to long queue times
- LLM-powered analysis can turn raw cluster data into actionable scheduling advice

Background

HPC clusters are used to run heavy computing tasks, especially machine learning and GPU jobs. Users often need SLURM commands like sinfo, squeue, scontrol, sbatch, and scancel to check resources and manage jobs.

However, it is not easy for new users to choose the right resources. Since cluster status changes often, users also need to understand SLURM outputs and error messages before submitting or fixing a job.

This motivates our assistant, which collects real-time cluster information, understands requests, and gives practical scheduling suggestions.

Related Work

The traditional way is to use commands such as sinfo, squeue, scontrol, sbatch, and scancel. However, users still need to manually interpret the output and decide how to configure their jobs.

Some web dashboards can visualize cluster usage and job status, but they usually cannot provide direct guidance for ambiguous user questions, such as “Which GPU should I use?” or “What GPU type fits this task’s requirements?”

Recent LLM-based agents can use tools and external APIs to answer complex questions, but they are not directly designed for UVA SLURM scheduling workflows. Our work combines real-time UVA SLURM data collection, LLM-based tool use, historical usage analysis, and human-approved job actions into one assistant for HPC scheduling support.

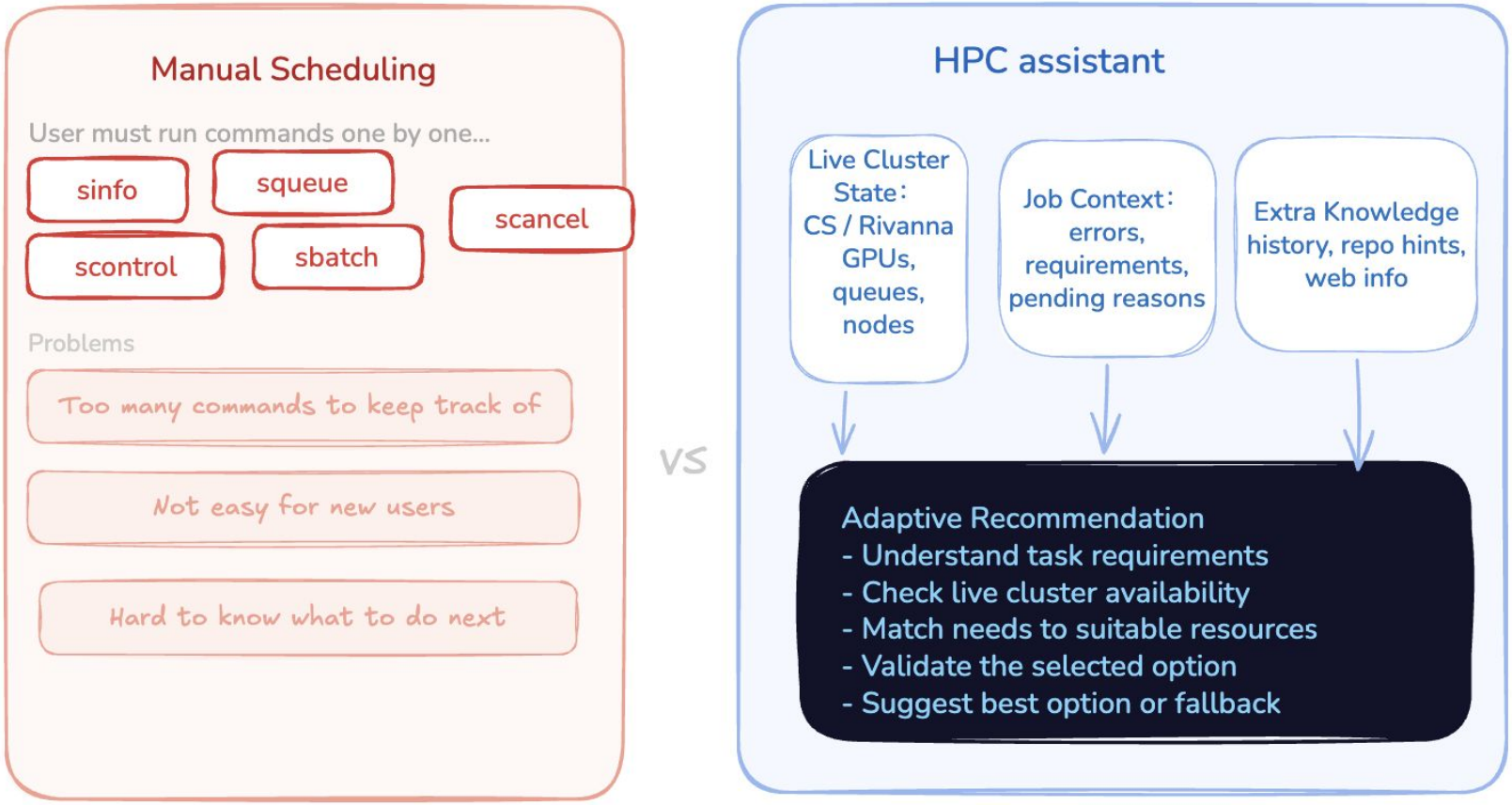
Claim / Target Task

The Objective:
Help users choose the right HPC resources for their SLURM jobs by analyzing project context, live cluster status, and resource requirements.

The Input:
User request + runtime context + current CS / Rivanna cluster state.

The Workflow:
Analyze project needs → check live cluster availability → recommend suitable resources (SLURM Operation option: → create a SLURM ticket when requested → wait for user approval before submission.)

An Intuitive Figure Showing WHY Claim

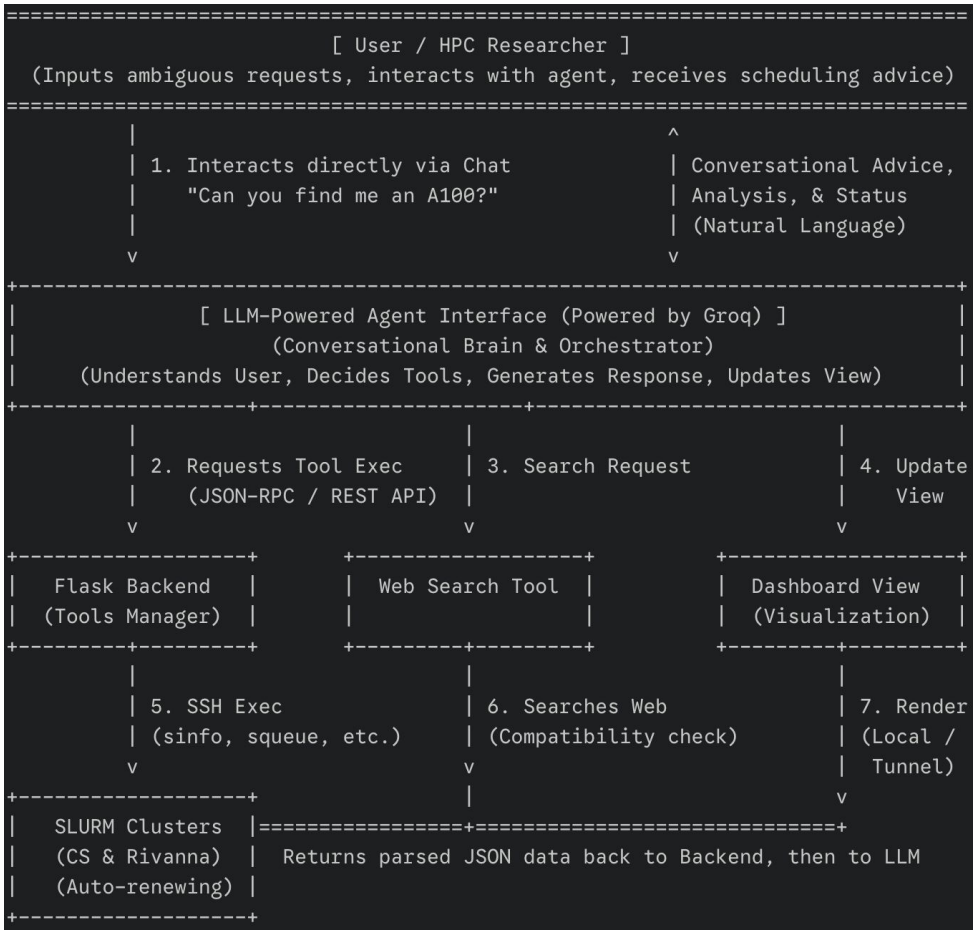


Proposed Solution

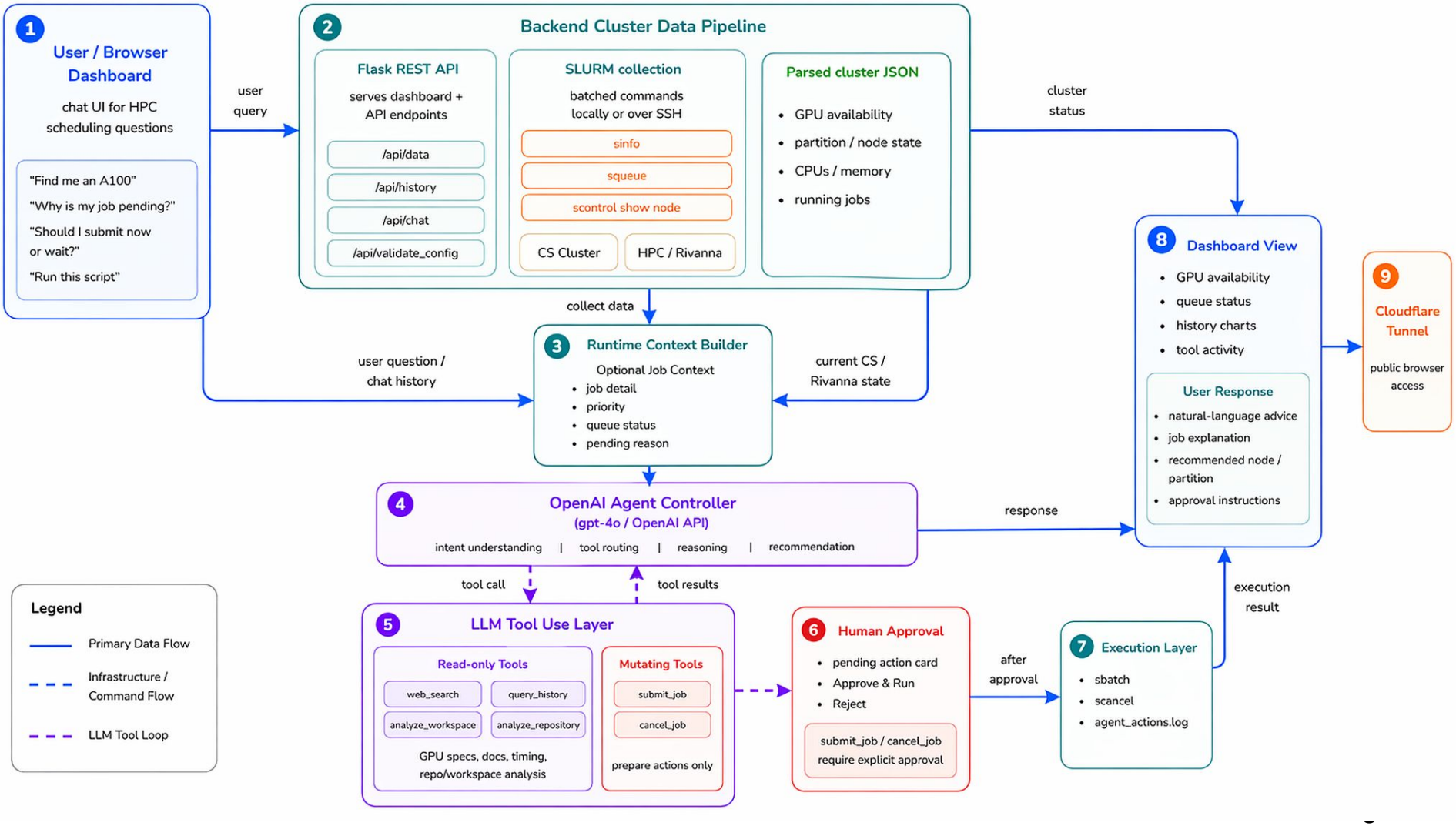
Data Collection: Batch SLURM commands (sinfo, squeue, scontrol) over SSH for multi-cluster support
Backend: Flask REST API serving cluster data as JSON

LLM for Ambiguous Requests: GPT 4-o integration for job queue analysis, SLURM request and scheduling recommendations

Infrastructure: Auto-renewing SLURM jobs + Cloudflare Tunnel for temporarily public access



Implementation



User Cases #1 Resource Recommendation

User Input:

“The baseline for my project is at https://github.com/joonspk-research/generative_agents.Which cluster GPU should I use?”

Agent Reasoning & Action:

1. Detects GitHub repo + resource recommendation request.

2. Calls *analyze_repository* tool.

3. Based on the given project’s resource requirements, Finds mostly lightweight / CPU-oriented dependencies.

4. Uses live cluster state to avoid unnecessary high-end GPU usage.

5. Recommends a suitable available node and SLURM resource settings.

Tool Used:

analyze_repository

Agent Result:

The project does not clearly require a high-end GPU.

Recommended option:

CS Cluster, udc-an28-18, RTX 3090, partition=gpu, cpus=4, mem=16G, gpus=1.

User Cases #2 SLURM operation

User Input:

"Can you help me submit a short GPU job on the CS cluster?I need 1 A100 for 1 hour, and the command is: python /home/tec7ws/Q/monitor.py"

Agent Reasoning & Action:

1. Detects a direct SLURM job submission request.
2. Extracts requirements: CS cluster, 1 A100, 1 hour, command path.
3. Calls *submit_job* to prepare a SLURM job ticket.
4. Shows an approval card instead of submitting immediately.
5. After user approval, runs sbatch and submits the job.

Tool Used:

submit_job

Agent Result:

The job is submitted successfully after approval:

Fallback:

If the requested A100 is occupied, the agent recommends another suitable and available resource based on the job needs and live cluster status.

Video Demo

https://youtu.be/2ZRQGhrS1Nc

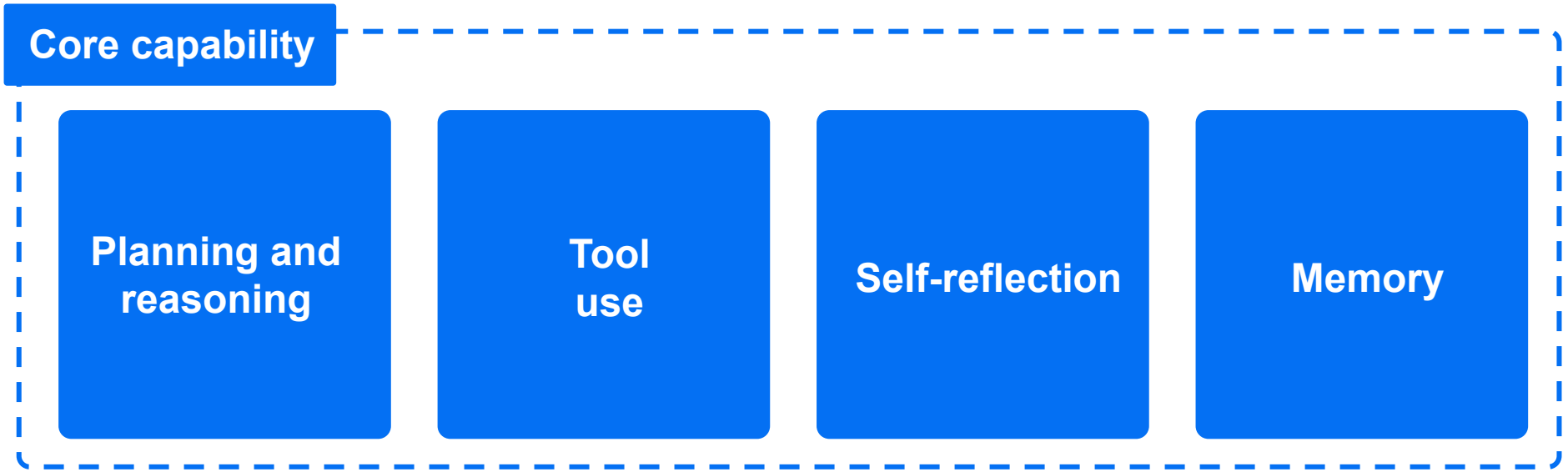
How to evaluate LLM agents?

Dimension	Project Setting
Interaction Mode	Dynamic data**.** The evaluation tests the live dashboard and live CS / Rivanna cluster state
Evaluation Data	The script reads dashboard data from /api/data/<cluster id>, collects ground truth using direct sinfo, and samples auto-renewal / tunnel status.
Metrics Computation Methods	The script computes correctness deltas, PASS/FAIL checks, min / p50 / p95 / max / mean latency, uptime percentage, longest outage, and recovery status.
Evaluation Tooling	Custom **evaluation.py** test. It includes three tests: correctness, latency, and reliability. It uses HTTP requests, direct SLURM commands, SQLite/state files, and tunnel checks.
Evaluation Contexts	Live HPC environment

How to evaluate LLM agents?

Metric	What It Measures	How It Is Tested
Correctness	Whether dashboard cluster data matches SLURM ground truth.	Compare /api/data/<cluster id> against independent sinfo results for total GPUs, free GPUs, total CPUs, free CPUs, and total GPU nodes.
Latency	Whether the dashboard refreshes fast enough for real-time use.	Send repeated GET requests to /api/data/<cluster id> and compute min, p50, p95, max, and mean response time. Target: p95 < 5s.
Reliability	Whether the system stays available over time.	Sample the auto-renewal job chain and Cloudflare tunnel availability. PASS requires chain uptime ≥ 95% and tunnel uptime ≥ 95%.

What to evaluate for Agentic AI System?



Understands job needs and makes a scheduling plan.

Understands job needs and makes a scheduling plan.

Checks whether the selected node, partition, and GPU are valid before recommending or submitting.

Uses chat history and past cluster availability to improve recommendations.

Data Summary

Clusters tested: CS & Rivanna cluster

Correctness data: dashboard API output vs direct SLURM ground-truth query(get via commands like *sinfo*)

Latency data: 10 dashboard API requests per cluster

Reliability data: 20 samples over 5 minutes, sampled every 15 seconds

Reliability target: auto-renewal chain uptime ≥ 95%, tunnel uptime ≥ 95%

Experimental Results

Correctness:

Dashboard: http://udc-aw36-17c1:8080
Tolerance: ±2 GPUs, ±4 CPUs (per cluster)

Dashboard fetched in 0.52s, ground truth in 0.05s (skew window ~0.6s)

```
  metric                dashboard  ground_truth    delta  result
-------------------------------------------------------------------
  total_gpus                  456           456       +0  OK
  free_gpus                    98            98       +0  OK
  total_cpus                 4632          4632       +0  OK
  free_cpus                  2791          2791       +0  OK
  total_gpu_nodes              54            54       +0  OK
  CLUSTER RESULT: PASS
```

Latency:

10 samples per cluster, target p95 < 5.0s

Dashboard: http://udc-aw36-17c1:8080

```
--- CS (cs) ---
.....
  samples ok=10/10 errors=0
  min=1.07s  p50=1.40s  p95=2.22s  max=2.65s  mean=1.48s
  raw sinfo floor (1 sample): 0.28s — dashboard overhead ≈ +1.12s
  RESULT: PASS (p95 < 5.0s)

--- Rivanna (hpc) ---
.....
  samples ok=10/10 errors=0
  min=0.25s  p50=0.62s  p95=0.76s  max=0.77s  mean=0.56s
  raw sinfo floor (1 sample): 0.59s — dashboard overhead ≈ +0.03s
  RESULT: PASS (p95 < 5.0s)

OVERALL LATENCY: PASS
```

Reliability

sampling for 5 min every 15s

```
  # time      chain_status  r/p   tunnel    detail
  1 17:26:06  healthy       1/1   ok        200 in 0.35s
  2 17:26:22  healthy       1/1   ok        200 in 0.14s
  3 17:26:37  healthy       1/1   ok        200 in 0.13s
  4 17:26:53  healthy       1/1   ok        200 in 0.11s
  5 17:27:09  healthy       1/1   ok        200 in 0.23s
  6 17:27:24  healthy       1/1   ok        200 in 0.15s
  7 17:27:40  healthy       1/1   ok        200 in 0.13s
  8 17:27:55  healthy       1/1   ok        200 in 0.12s
  9 17:28:11  healthy       1/1   ok        200 in 0.22s
 10 17:28:26  healthy       1/1   ok        200 in 0.37s
 11 17:28:42  healthy       1/1   ok        200 in 0.13s
 12 17:28:57  healthy       1/1   ok        200 in 0.11s
 13 17:29:12  healthy       1/1   ok        200 in 0.33s
 14 17:29:28  healthy       1/1   ok        200 in 0.20s
 15 17:29:44  healthy       1/1   ok        200 in 0.10s
 16 17:29:59  healthy       1/1   ok        200 in 0.14s
 17 17:30:15  healthy       1/1   ok        200 in 0.46s
 18 17:30:30  healthy       1/1   ok        200 in 0.13s
 19 17:30:46  healthy       1/1   ok        200 in 0.14s
 20 17:31:01  healthy       1/1   ok        200 in 0.14s

--- Auto-renewal chain ---
  samples:          20
  healthy (R+P+dep): 20 (100.0%)

--- Tunnel availability (passive) ---
  samples:          20
  uptime:           100.0%
  failures:         0
  longest outage:   0 consecutive samples (0s)
```

OVERALL RELIABILITY: PASS

Experimental Analysis

Correctness: PASS. correct for GPU and node statistics

Latency: PASS. Both CS and Rivanna respond within the 5-second p95 target.

Reliability: PASS. Auto-renewal and tunnel access stayed healthy during the test window.

Conclusion and Future Work

- the dashboard make the resource viewable.
- It combines project context, live cluster state, and LLM tool use to recommend suitable resources.
- It supports safe job operations through user-approved SLURM job tickets.
- Our future work is to add saved job templates, so users can quickly rerun common experiments without rewriting SLURM settings every time.

References

[1] Cloudflare. Cloudflare Tunnel Documentation. https://developers.cloudflare.com/tunnel/

- A page on who did what in your team

Guangyi Xu: Backend data collection, SLURM integration, shell scripts, features update, evaluation

Jinwei Zhou: Flask API, web dashboard frontend, testing, documentation

Wentao Zhou: LLM agent integration (gpt 4o), prompt design, evaluation, Infrastructure (Cloudflare tunnel, auto-renewal)